Containers Quiz Practice Problems

Note 1. A *container* is a class that holds other values. The built-in containers you know are `list`, `dict`, `set`, and `tuple`, and they all implement the same small set of dunder methods—the *container protocol*—so that the same syntax works on all of them. In this notes packet we implement the container protocol on types we define ourselves: a `Range`, a linked list, and a binary search tree.

Note 2. Each problem is worth two points. If the code prints two lines, then each print statement is worth one point. If the code prints only one line (due to an error), then that one line is worth two points.
Within each subsection the first problem is a correct reference implementation. The remaining problems contain subtle bugs or variations, so do not assume the code behaves like the corresponding built-in type—trace the code that is actually written.

1 Example: Range

Note 3. The built-in `range` function does not actually return a list, but an instance of the `range` class. See:

```
>>> type(range(10))
<class 'range'>
>>> range(10)
range(0, 10)
```

In this section we see how they implement some of the container-like features of this class.

```
>>> len(range(10))     # implemented by __len__ method
10
>>> 5 in range(10)     # implemented by __contains__ method
True
>>> range(10)[0]       # implemented by __getitem__ method
0
```

Problem 4. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return max(0, -((self.start - self.stop) // self.step))

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i < 0:
            i += n
        if i < 0 or i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(0, 10, 2)
    n = len(r)
    print('n =', n)
    x = 5 in r
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $15 \;/\; 16 = 0.94$

Problem 5. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return (self.stop - self.start) // self.step

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i < 0:
            i += n
        if i < 0 or i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(0, 50, 7)
    n = len(r)
    print('n =', n)
    x = -7 in r
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $10 / 16 = 0.62$

Problem 6. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return (self.stop - self.start) // self.step

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(0, 100, 7)
    n = len(r)
    print('n =', n)
    x = r[-1]
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 7. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return max(0, -((self.start - self.stop) // self.step))

    def __contains__(self, x):
        return self.start <= x < self.stop

    def __getitem__(self, i):
        n = len(self)
        if i < 0:
            i += n
        if i < 0 or i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(0, 10, 2)
    n = len(r)
    print('n =', n)
    x = 5 in r
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 8. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return -((self.start - self.stop) // self.step)

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i < 0:
            i += n
        if i < 0 or i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(5, 0)
    n = len(r)
    print('n =', n)
    x = 3 in r
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

Problem 9. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return max(0, -((self.start - self.stop) // self.step))

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i < 0 or i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(0, 10, 2)
    n = len(r)
    print('n =', n)
    x = r[-1]
    print('x =', x)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 10. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return max(0, -((self.start - self.stop) // self.step))

    def __contains__(self, x):
        if (x - self.start) % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i < 0:
            i += n
        if i < 0 or i >= n:
            raise ValueError(i)
        return self.start + self.step * i

try:
    r = Range(0, 10, 2)
    n = len(r)
    print('n =', n)
    x = r[10]
    print('x =', x)
except IndexError:
    print('IndexError')
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 11. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step

    def __len__(self):
        return max(0, -((self.start - self.stop) // self.step))

    def __contains__(self, x):
        if x % self.step != 0:
            return False
        return 0 <= (x - self.start) // self.step < len(self)

    def __getitem__(self, i):
        n = len(self)
        if i < 0:
            i += n
        if i < 0 or i >= n:
            raise IndexError(i)
        return self.start + self.step * i

try:
    r = Range(1, 10, 2)
    n = len(r)
    print('n =', n)
    x = 3 in r
    print('x =', x)
except ValueError:
    print('ValueError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $12 \;/\; 16 = 0.75$

2 Structural Recursion

Note 12. *Structural recursion* is when a class has recursive structure. It is very common for container-like classes to use structural recursion.

Note 13. Every structurally recursive class has the same two ingredients:

1. a *base case* for the smallest structure (often `None`), and
2. a *recursive attribute* that points to another instance of the same class.

2.1 Linked Lists

Note 14. A *linked list* is the simplest of recursive structures. The `Node` class below has an attribute `nxt` that points to `None` (the base case) or another `Node` instance (the recursive case). The linked lists in this section implement an interface similar to the `[]` list type.

Problem 15. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = xs[2]
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 16. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 0
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.nxt is None:
            return False
        if self.val == target:
            return True
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = 4 in xs
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $12 / 16 = 0.75$

Problem 17. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(5, Node(10, Node(15)))
    n = len(xs)
    print('n =', n)
    v = 20 in xs
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $14 \;/\; 16 = 0.88$

Problem 18. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 0
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = xs[2]
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 19. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = 3 in xs
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $9 / 16 = 0.56$

Problem 20. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = xs[2]
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $8 / 16 = 0.50$

Problem 21. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = 9 in xs
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 22. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.nxt is None:
            return False
        if self.val == target:
            return True
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    n = len(xs)
    print('n =', n)
    v = 4 in xs
    print('v =', v)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $14 / 16 = 0.88$

Problem 23. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    xs.nxt.nxt.nxt.nxt = xs
    v = xs[6]
    print('v =', v)
    n = len(xs)
    print('n =', n)
except IndexError:
    print('IndexError')
except RecursionError:
    print('RecursionError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 24. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __len__(self):
        if self.nxt is None:
            return 1
        return 1 + len(self.nxt)

    def __contains__(self, target):
        if self.val == target:
            return True
        if self.nxt is None:
            return False
        return target in self.nxt

    def __getitem__(self, i):
        if i == 0:
            return self.val
        if self.nxt is None:
            raise IndexError(i)
        return self.nxt[i - 1]

try:
    xs = Node(1, Node(2, Node(3, Node(4))))
    xs.nxt.nxt.nxt.nxt = xs
    v = xs[-1]
    print('v =', v)
    n = len(xs)
    print('n =', n)
except IndexError:
    print('IndexError')
except RecursionError:
    print('RecursionError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

2.2 Binary Search Trees

Note 25. A *tree* is like a linked list, but has two recursive attributes instead of one. A *binary search tree* (BST) satisfies the invariant that: every node in the left branch is less than or equal to val, and every node in the right branch is greater than or equal to val. This property lets us *prune* branches of the tree when doing searches, and makes searching much faster.

Problem 26. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.left = insert(t.left, val)
    elif val > t.val:
        t.right = insert(t.right, val)
    return t

def inorder(t):
    if t is None:
        return []
    return inorder(t.left) + [t.val] + inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return inorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[1]
    print('x =', x)
    y = t[-1]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 27. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.right = insert(t.right, val)
    elif val > t.val:
        t.left = insert(t.left, val)
    return t

def inorder(t):
    if t is None:
        return []
    return inorder(t.left) + [t.val] + inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return inorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[1]
    print('x =', x)
    y = t[3]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 28. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    t.right = insert(t.right, val)
    return t

def inorder(t):
    if t is None:
        return []
    return inorder(t.left) + [t.val] + inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return inorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[2]
    print('x =', x)
    y = t[-2]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 29. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.left = insert(t.left, val)
    elif val > t.val:
        t.right = insert(t.right, val)
    return t

def preorder(t):
    if t is None:
        return []
    return [t.val] + preorder(t.left) + preorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return preorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[0]
    print('x =', x)
    y = t[2]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 30. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.left = insert(t.left, val)
    elif val > t.val:
        t.right = insert(t.right, val)
    return t

def postorder(t):
    if t is None:
        return []
    return postorder(t.left) + postorder(t.right) + [t.val]

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return postorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[1]
    print('x =', x)
    y = t[-1]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 31. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        insert(t.left, val)
    elif val > t.val:
        insert(t.right, val)
    return t

def inorder(t):
    if t is None:
        return []
    return inorder(t.left) + [t.val] + inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return inorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[0]
    print('x =', x)
    y = t[1]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 32. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.left = insert(t.left, val)
    elif val > t.val:
        t.right = insert(t.right, val)
    return t

def inorder(t):
    return inorder(t.left) + [t.val] + inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return inorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 1, 4]:
        t.insert(val)
    x = t[0]
    print('x =', x)
    y = t[2]
    print('y =', y)
except AttributeError:
    print('AttributeError')
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 33. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(t, val):
    if t is None:
        return Node(val)
    if val < t.val:
        t.left = insert(t.left, val)
    elif val >= t.val:
        t.right = insert(t.right, val)
    return t

def inorder(t):
    if t is None:
        return []
    return inorder(t.left) + [t.val] + inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = insert(self.root, val)

    def __getitem__(self, i):
        return inorder(self.root)[i]

try:
    t = Tree()
    for val in [5, 3, 8, 3]:
        t.insert(val)
    x = t[0]
    print('x =', x)
    y = t[3]
    print('y =', y)
except IndexError:
    print('IndexError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

2.3 Weird Problems

Note 34. All of the previous problems have an intuitive application, and so if the problem is implemented correctly you can "guess" what the correct answer should be without manually working through the recursion. The problems below all have no corresponding intuition, and there is no easy way to guess for correct solution should be without walking through the computation.

Problem 35. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self, n):
        if n == 0:
            return 0
        return self.x + self.y.wob(n - 1)

t = FooBar(5)
t.y = t
try:
    print(t.y.y.x)
    print(t.wob(4))
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 36. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(7)
t.y = t
try:
    print(t.y.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 37. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None, z=None):
        self.x = x
        self.y = y
        self.z = z

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(1)
u = FooBar(2)
t.y = u
u.y = t
t.z = 5
try:
    print(t.z.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 38. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(1)
u = FooBar(2)
v = FooBar(3)
t.y = u
u.y = v
v.y = t
try:
    print(t.y.y.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 39. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y if y is not None else self

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(10)
try:
    print(t.y.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 40. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None, z=None):
        self.x = x
        self.y = y
        self.z = z

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(1)
u = FooBar(2)
t.y = u
t.z = t
try:
    print(t.z.z.z.z.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 41. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(42)
t.y = t
try:
    print(t.y.y.y.y.y.y.y.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 42. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self, n):
        if n == 0:
            return 0
        return self.x + self.y.wob(n - 1)

t = FooBar(1)
u = FooBar(2)
v = FooBar(3)
t.y = u
u.y = v
v.y = t
try:
    print(t.y.y.x)
    print(t.wob(6))
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 43. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class FooBar:
    def __init__(self, x, y=None):
        self.x = x
        self.y = y

    def wob(self):
        if self.y is None:
            return self.x
        return self.x + self.y.wob()

t = FooBar(None)
u = FooBar(5)
t.y = u
try:
    print(t.y.x)
    print(t.wob())
except RecursionError:
    print('RecursionError')
except AttributeError:
    print('AttributeError')
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $12 \ / \ 16 = 0.75$

LLM Model Performance

